Senior / Lead .NET & Azure — 2-Page Last-Minute Review

Read on the way in. Full dossier has the depth; this is the recall layer.

Cheat Sheet

.NET / C#

- Async frees the thread at I/O; no sync-context in Core.
- Starvation = sync-over-async (`.Result`) → go async end-to-end.
- `ValueTask` only on hot, sync-completing paths; await once.
- LOH >85KB → pool buffers; Server GC for throughput.
- `Span<T>` for transient slicing; no boxing in hot loops.
- Middleware: exception→correlation→authN→authZ→tenant→endpoint.

SQL Server

- Diagnose from actual plan + Query Store, not a rewrite.
- Fast→slow = stale stats or parameter sniffing.
- Deadlock: canonical lock order + covering index; retry is defense only.
- Composite = Equality, Range, Include (covering kills lookups).
- Optimistic (`rowversion`) default; pessimistic for tiny critical sections.

React

- Split server-state (React Query) from client-state (RTK/Zustand).
- Context = DI, not a store; split providers.
- Re-renders: Profiler + stable refs + virtualization, not blanket `useMemo`.
- Tokens in `HttpOnly` cookie / BFF; never localStorage.

Cloud / Architecture

- Exactly-once effect = at-least-once + idempotent consumers + outbox.
- Service Bus (ordered commands) vs Event Hubs (volume events).
- Tenancy: silo / pool (+RLS) / bridge; hybrid is fine.
- Zero-downtime: Blue/Green slots + Canary, expand/contract schema.
- Zero Trust: verify explicitly, least privilege, assume breach.
- Resilience: Polly timeout + retry-jitter + breaker + bulkhead.

Leadership (STAR spine)

- Scope creep → capacity map + thin slice + options w/ trade-offs.
- Release block → impact/options/recommendation/ask; shield team.
- Conflict → ADR + CI architecture tests depersonalize it.
- Close with a metric + "what I'd do differently."

SRE / Delivery

- Pyramid: Unit → Integration (Testcontainers) → thin E2E (Playwright).
- SLO + error budget drives ship-vs-freeze.
- OTel tracing cuts MTTR; DORA tracks the delivery system.
- Auto-rollback on SLI breach; every change flagged + reversible.

AI / Cursor

- Force multiplier; accountable for every merged line.
- Composer refactors + `.cursorrules` + reviewed AI tests.
- Audit diffs: leaks, sync-over-async, SQL injection, secrets.

Enterprise stances (one-liners)

- **Monolith→microservices?** Only if teams/deploys are truly coupled. Modular monolith first; never shared DB.
- **Zero-downtime migration?** Strangler-fig + expand/contract + dual-write/shadow-read, cut over behind a flag.
- **Flaky third-party?** Polly breaker + bulkhead + cache/fallback; degrade, don't cascade.
- **Security blocks release?** Stakeholder not gate: quantify risk, compensating controls, remediation timeline.

Top 10 Questions — Answer Spine

ASYNC **1. Why does async scale under load?**
Thread returns to pool at I/O (IOCP resumes continuation); few threads serve thousands of I/O-bound requests. No sync-context in Core. Killer = sync-over-async → pool starvation (p99 climbs, CPU idle). Nothing gained for CPU-bound work.

VALUETASK **2. `ValueTask` vs `Task`?**
`Task` heap-allocates per call; `ValueTask` avoids it on sync-completing hot paths. Await exactly once, no double-await, no stored-await, no early `.Result`; else `.AsTask()`. Default `Task`; `ValueTask` only when a profiler proves it.

GC **3. Latency spikes + rising memory → GC?**
Confirm via counters (Gen2/LOH, %time in GC, alloc rate). LOH >85KB → pool buffers/stream; Gen0 churn → `Span` / `stackalloc`; wrong mode → Server GC; mid-life leaks → bounded cache/weak refs. Trade-off: Server GC costs memory on constrained containers.

SQL LATENCY **4. Fast-before, slow-now, same query?**
Read the actual plan + Query Store first. Usually stale stats or parameter sniffing; also blocking or a missing covering index as data grew. Bring the Query Store regression artifact, not a guess.

DEADLOCKS **5. Recurring deadlocks under heavy writes?**
Capture the deadlock graph. Fix the cause: canonical lock ordering, covering index (seek not scan), shorter transactions. Retry-on-1205 with jitter is defense-in-depth; consider RCSI (watch tempdb).

INDEXING **6. Composite index column order?**
Equality, Range, then Include. Leading = most-selective equality columns; select-only columns go in `INCLUDE` for a covering index (no key lookups). Audit unused/duplicate — every index costs on writes.

CONCURRENCY **7. Optimistic vs pessimistic?**
Optimistic (`rowversion` in WHERE) default for read-heavy contention — no locks across think-time. Pessimistic only for tiny high-value critical sections. Money movement adds an idempotency key (separate concern).

REACT TOKENS **8. Where to store the SPA token?**
`HttpOnly` + `Secure` + `SameSite` cookie, ideally via a BFF holding the real OAuth tokens server-side — browser never sees a bearer. localStorage = XSS exfil. Cookie cost is CSRF → SameSite + anti-forgery + CSP.

SYSTEM DESIGN **9. Event-driven multi-tenant transaction system, 10k TPS?**
Front Door→APIM (JWT, rate-limit)→thin intake assigns idempotency key→Service Bus (sessions for order)→idempotent Functions (dedup store)→Azure SQL + transactional outbox→saga for cross-service. Tenancy pool+RLS / silo hybrid. Blue/Green + Canary, OTel tracing. "At-least-once + idempotent + outbox = exactly-once effect."

ZERO TRUST **10. Zero Trust concretely?**
Verify explicitly (managed identity + validated JWTs at APIM and service); least privilege (scoped RBAC + row-level security); assume breach (private endpoints, Key Vault rotation, encryption, audit logs). Identity is the boundary, not the network.

Questions to ask them

- On-call culture — blameless postmortems, who owns MTTR? · This lead's technical authority vs the EM's people authority? · Biggest architectural regret + appetite to pay it down? · Platform/shared vs squad-owned boundaries? · Current change-failure rate & lead time, and the one-year target?

2-page recall layer. Lead with the decision, name the trade-off, close with a metric + "what I'd do differently." Calibrate all numbers to your real history.